

AWS Developer Certification – Domain 1 Study Summary

Domain 1: Development with AWS Services — Task Statement 1: *Develop Code for Applications Hosted on AWS*

Summary notes from an exam-prep training transcript. Practice questions called out in the transcript appear as blockquotes throughout, with a quick-reference answer key at the end.

Foundations to Review

- **AWS Well-Architected Framework** – focus on the **Operational Excellence** pillar: supporting development, running workloads effectively, gaining operational insight, and continuously improving processes. Apply the same discipline you use for application code to your whole environment — e.g., automating deployments via event triggers, and sharing improvements across teams.
- **AWS CloudFormation** and **AWS SAM (Serverless Application Model)** — both recur throughout this task statement.

Design Patterns: Know the Workload First

Before writing code, nail down how the app will be accessed, its usage patterns, whether hardware needs managing, its development/deployment patterns, and how it functions end-to-end.

Event-driven architecture

- Know the difference between **tightly-coupled** and **loosely-coupled** components — a recurring exam theme.
- Event-driven design decouples services and underlies microservices; most AWS services can generate or act as event sources (Lambda is a key example).
- Benefits: scalable, resilient, agile, cost-effective.
- Core services: **SQS, SNS, EventBridge**.
- Trade-off vs. monolithic apps: a monolith processes everything in one memory space on one device; an event-driven app communicates across the network, so components can fail independently — code needs to detect and automatically remediate those failures.

Building Resilience Into Application Code

Exponential backoff with jitter

- Increases reliability and can lower operational cost.
- Retrying immediately isn't always safe — it can pile more load onto an already-struggling system.

- Use **exponential backoff** (progressively longer waits between retries); add **jitter** if backoff alone still causes contention.

Handling message failures

- Build in **destinations** and **dead-letter queues (DLQs)**.
- Lambda can route invocation results to SQS, EventBridge, or SNS.
- **Step Functions** can separate out retries, backoff rates, max attempts, intervals, and timeouts.

Orchestration

- Workflows with branching logic, multiple failure types, and retries need something tracking overall state — an orchestrator.
- Start with Lambda or messaging patterns; migrate to a state machine (**Step Functions**) as complexity grows.
- Step Functions handles nested workflows, errors, and retries; executions can run **up to a year** and support versioning (easier upgrades, less custom code).
- When you need to coordinate state changes **across multiple services**, **EventBridge** may fit better than Step Functions.

Fanout pattern

- Publish one message to multiple endpoints (SQS, Data Firehose, HTTPS endpoints, Lambda).
- Common use case: replicate production data into a test environment.

Idempotency & data integrity

- Validate input messages/calls/events and track whether an event has already been processed.
- Add **idempotent logic** to Lambda so repeated/retried requests have no additional effect — this cuts down on duplicate API calls, inconsistency, throttling, and latency.

Synchronous vs. Asynchronous, Stateless vs. Stateful

- **Step Functions** supports **both** synchronous and asynchronous workflows.
- **REST APIs** are typically synchronous — a response is expected.
- If a service crashes mid-synchronous-processing, in-flight data is normally lost, unless the design explicitly persists the incoming message first (e.g., writing it to **SQS**, which retains messages).
- **Stateless vs. stateful**: both store client state server-side and use a database as the backend, but a **stateful** app skips a second database call on a client's follow-up request, improving performance. Know how each approach scales.

- **Containers are typically stateless**, which supports scalability and high availability, since orchestration and load balancing can add instances as demand grows.

Practice question — ECS port mapping: Building a Docker app on ECS, and containers need to access ports on the host container instance for traffic. Which ECS component do you configure — the container agent, the container instance, the task definition, or the agent?

Answer: the task definition — port mappings live in the container definition, which is configured in the task definition.

AWS SDKs & Serverless Tooling

- SDKs reduce the complexity of authentication, retries, and error handling — get comfortable with the official SDK/service code examples.
- Know microservice design patterns spanning **ECS, Lambda, Cognito, API Gateway, CloudFront, DynamoDB, and S3**.

Practice question — choosing a deployment service: Migrating to AWS and building a new serverless architecture where Lambda, API Gateway, and DynamoDB share memory/timeouts and deploy together as a single versioned application. Options: CloudFormation, OpsWorks, Elastic Beanstalk, or SAM?

Answer: AWS SAM — CloudFormation could technically do it, but SAM is purpose-built to simplify defining the API Gateway APIs, Lambda functions, and DynamoDB tables a serverless app needs, as one unit.

API Gateway — Expect Heavy Exam Coverage

Know in depth:

- Overriding status codes, validation rules, response transformations
- Method requests/responses vs. integration requests/responses
- Mapping templates
- Stages and stage variables
- Caching and throttling
- Usage plans and API keys

Practice question — non-proxy integration: An online school app uses a non-proxy Lambda integration. A new GET method on a course resource needs a required query string parameter to return the course list. Do you configure this on the method request or the method response?

Answer: the method request — that's where the required query parameter is defined.

Streaming Data

- Managed option: **Amazon Kinesis**; or self-manage on EC2.

- Plan for scalability, durability, and fault tolerance across both the storage and processing layers.
- Platforms to know: **Kinesis Data Streams, Amazon Data Firehose, Amazon MSK (Managed Streaming for Kafka), Apache Flume, Apache Spark Streaming, Apache Storm.**

Development Lifecycle

- **Develop** — write code, pick tooling, source repo, data storage, integrate services.
- **Build** — compilation, resource generation, packaging, storing build artifacts (dive deeper into **CodeBuild**).
- **Test** — write/run unit tests, automate test environments (dive deeper into **SAM, CodePipeline, CloudFormation**).

Practice question — syncing user data: An app is growing and needs to sync a user's profile data across their devices. Options: AWS Amplify, Cognito identity pools, Cognito user pools, or Cognito Sync?

Answer: Cognito Sync — a client library that syncs user profile data across a user's devices and the internet, without needing your own backend.

Practice question — DynamoDB hot partition: Reads/writes on a DynamoDB table are being throttled. CloudWatch shows consumed capacity hasn't exceeded provisioned capacity — investigation points to a **hot partition** getting disproportionately more traffic than the rest of the table. Options: increase capacity, implement sharding to distribute the workload, use DAX, implement retries with exponential backoff, or refactor to distribute reads/writes evenly.

Takeaway: since overall throughput isn't the constraint, raising provisioned capacity won't fix it — the real fix is redistributing access more evenly across partitions (sharding / a better partition key). The transcript separately flags retries with exponential backoff as good practice for reliability, noting it's **already built into the AWS SDK** (you'd only need to add that logic manually with a non-AWS SDK).

Broader Service List to Dive Deeper Into

For microservice architectures on AWS, be comfortable with: **SQS, SNS, Kinesis Data Streams, other Kinesis services, DynamoDB Streams, AWS IoT, Amazon MQ, and Step Functions.**

Up Next

Task Statement 2: Develop code for AWS Lambda.

Quick-Reference Answer Key

#	Scenario	Answer
1	ECS needs port mapping configured for host traffic	Task definition
2	Serverless app (Lambda + API Gateway + DynamoDB) deployed as one versioned unit	AWS SAM
3	API Gateway non-proxy integration needs a required query parameter	Method request
4	Sync user profile data across a user's devices	Cognito Sync
5	DynamoDB hot partition throttling, capacity not exceeded	Redistribute workload across partitions (sharding / partition key redesign)